Inheritance in Java

20 November 2025 20:27

Inheritance in Java is a mechanism in which one class acquires all the properties and behaviors of a parent class.
The idea behind inheritance in Java is that we can create new classes that are built upon existing classes. When we inherit methods from an existing class, we can reuse methods and fields of the parent class. However, we can add new methods and fields in your current class also.

What is Inheritance?
Inheritance in Java enables a class to inherit properties and actions from another class called a super class or parent class. OK a class derived from a super class is called a subclass or child class. Through inheritance, a subclass can access member of its superclass (fields and methods), enforce reuse rules and encourage hierarchy.

Inheritance represents the "**IS-A**" relationship, which is also known as a parent child relationship.

Why use inheritance?
- **For Method Overriding(so runtime polymorphism can be achieved).**
- **For code Reusability.**

Using inheritance we can add new features or remove redundant features of a software, without disturbing the business of the client.

Terms used in Inheritance
- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.

- **Sub class or Child class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class or child class.

- **Super class or parent class:** Super class is the class from where a subclass inherits the feature. It is also called a base class or a parent class.

- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class. When you create a new glass. You can use the same fields and methods already defined in the previous class.

The Syntax of Java Inheritance:

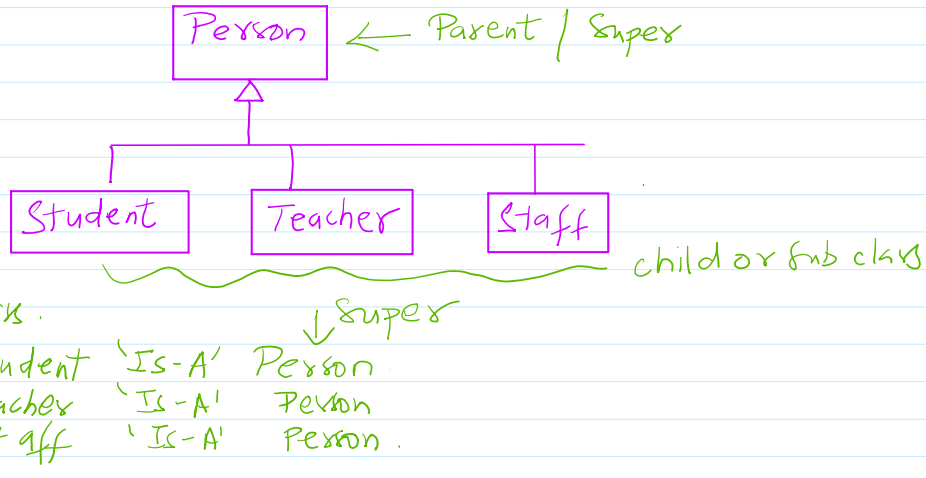
```
class Subclass-name extends Superclass-name{
  //methods and fields
}
```

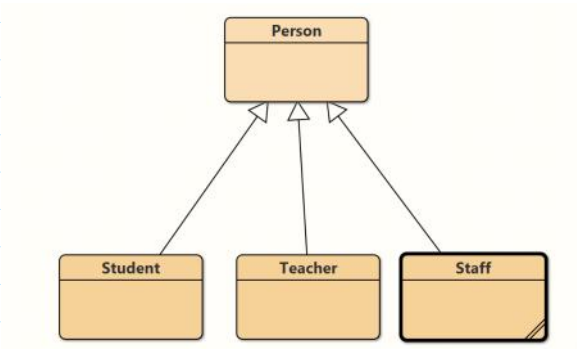
The **extend** keyword indicates that we are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

In the terminology of Java, a class that is inherited is called a parent or a super class and the new class is called child or sub class.

Person ← Parent / Super

class and the new class is called child or sub class.





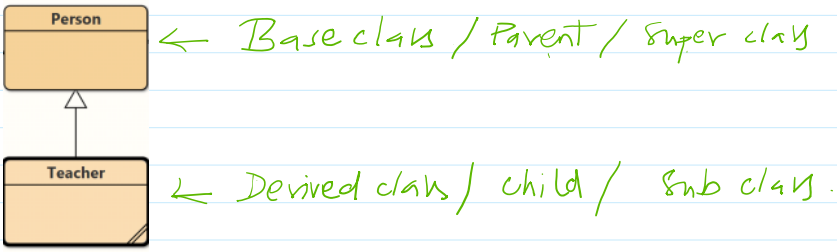
Inheritance diagram

Types of Inheritance in Java

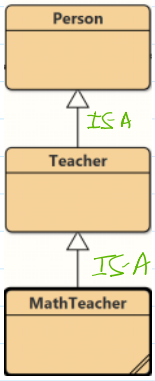
On the basis of class, there can be three types of inheritance in Java:

1) Single Inheritance
2) Multilevel Inheritance
3) Hierarchical Inheritance

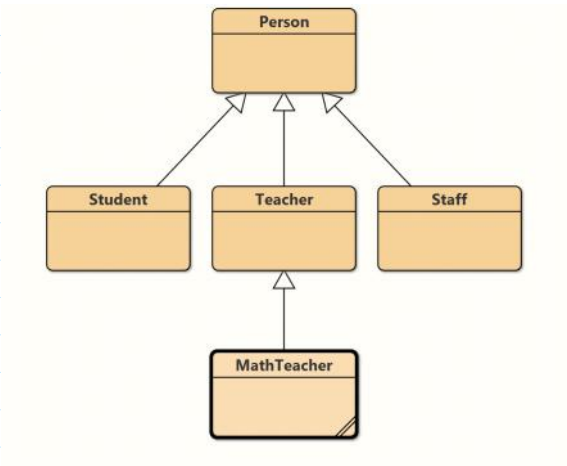
Single Inheritance: When a class inherits another class, it is known as single inheritance.



Multilevel Inheritance: When there is a chain of inheritance it is known as multilevel inheritance.

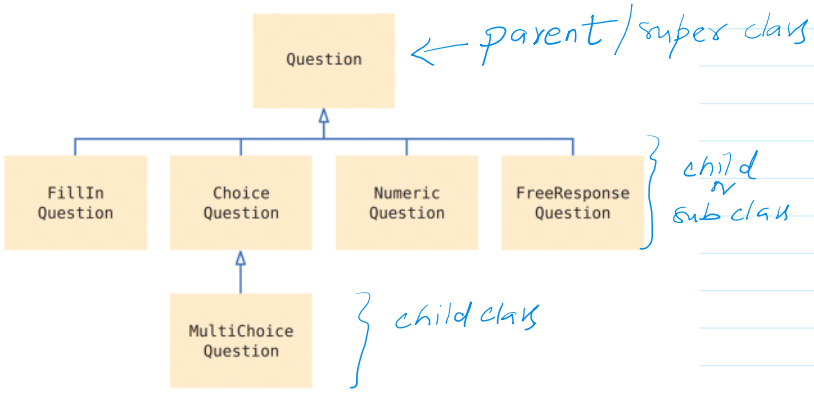


Hierarchical Inheritance: When two or more classes inherits a single class, it is known as hierarchical inheritance. Example is given below:



1972
1985
C++
Python
1990
1996-97
almost 30 years
JAVA
Kotlin.

For example:



"Java supports only 1 level of inheritance."

Choice Question

Question Portion

text =

answer =

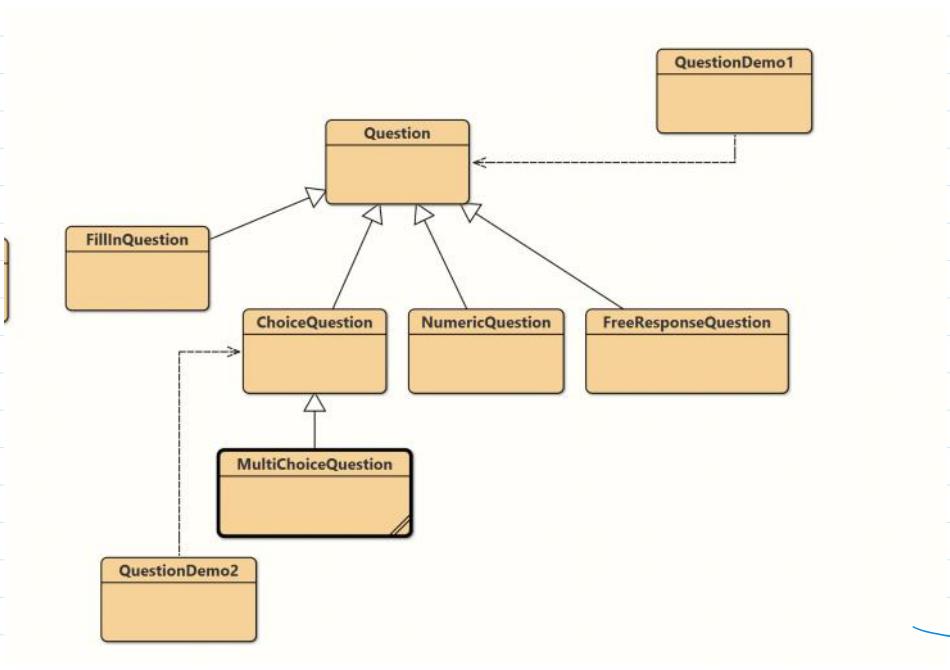
choices = [ ]

```
package Inheritance;


/**
 * Write a description of class Question here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
/**
 * A question with a text and an answer
 */
public class Question
{
    private String text;
    private String answer;

    /**
     * Constructs a question with empty question and answer
     */
    public Question(){
        text = answer = "";
    }

    /**
     * Sets the question text
     * @param questionText the of this question
     */
    public void setText(String questionText){
        text = questionText;
    }

    /**
     * Sets the answer for this question
     * @param correctResponse the answer
     */
    public void setAnswer(String correctResponse){
        answer = correctResponse;
    }

    /**
     * Checks a given response for correctness
     * @param response the response to check
     * @return true if the response was correct, false otherwise
     */
    public boolean checkAnswer(String response){
        return response.equalsIgnoreCase(answer);
    }

    /**
     * Display this question
     */
    public void display(){
```

```
        System.out.print("\n" + text);
    }
}

package Inheritance;
import java.util.*;


/**
 * Write a description of class QuestionDemo1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class QuestionDemo1
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        Question q = new Question();
        q.setText("Who was the inventor of Java?");
        q.setAnswer("James Gosling");

        q.display();
        System.out.print("\nYour answer: ");
        String ans = sc.nextLine();
        System.out.print("\n" + q.checkAnswer(ans));
    }
}


package Inheritance;
import java.util.*;


/**
 * Write a description of class ChoiceQuestion here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ChoiceQuestion extends Question
{
    private ArrayList<String> choices;

    /**
     * Constructs a choice question with no choices
     */
    public ChoiceQuestion(){
        choices = new ArrayList<String>();
    }

    /**
     * Adds an answer choice to this question
     * @param choice the choice to add
     * @param correct true if this is the correct choice, false otherwise
     */
    public void addChoice(String choice, boolean correct){
```

```
        choices.add(choice);
        if(correct){
            setAnswer(choice);
        }
    }
    @Override
    public void display(){
        super.display();
        //Display the answer choices
        for(int i = 0; i < choices.size(); i++){
            System.out.print("\n" + (i+1) + ": " + choices.get(i));
        }
    }
}

package Inheritance;
import java.util.*;


/**
 * Write a description of class QuestionDemo2 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class QuestionDemo2
{
    public static void main(String args[]){
        ChoiceQuestion first = new ChoiceQuestion();
        first.setText("What was the original name of the Java language?");
        first.addChoice("7*", false);
        first.addChoice("Duke", false);
        first.addChoice("Oak", true);
        first.addChoice("Gosling", false);

        ChoiceQuestion second = new ChoiceQuestion();
        second.setText("In which country was the inventor of Java born?");
        second.addChoice("Australia", false);
        second.addChoice("Canada", true);
        second.addChoice("Denmark", false);
        second.addChoice("United States", false);

        presentQuestion(first);
        presentQuestion(second);
    }

    /**
     * Presents a question to the user and checks the response
     * @param q the question
     */

    public static void presentQuestion(ChoiceQuestion q){
        q.display();
        System.out.print("\nYour answer: ");
        String response = new Scanner(System.in).nextLine();
        System.out.print("\n" + q.checkAnswer(response));
    }
```

}



String a[] = new String[5];

This array is growable and shrinkable on runtime as per our requirement

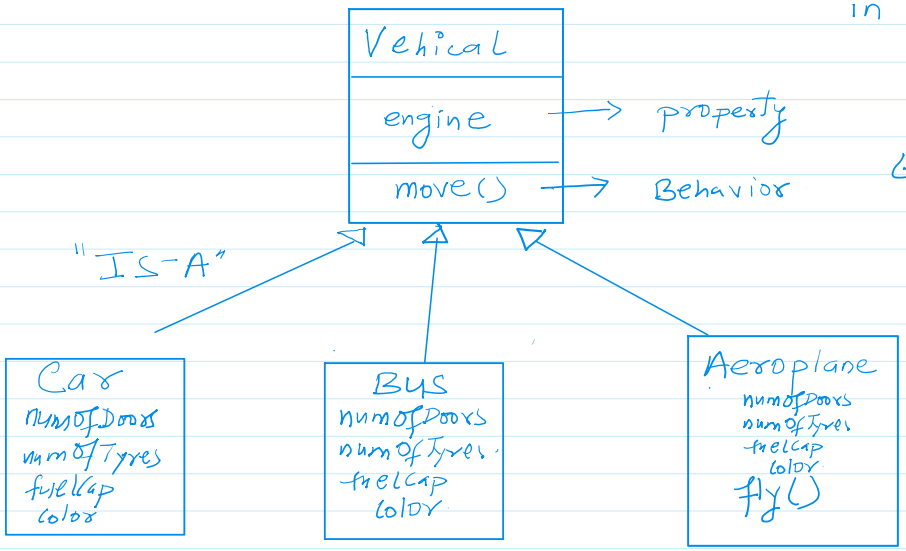
ArrayList

class

ArrayList<String> astr = new ArrayList<>();

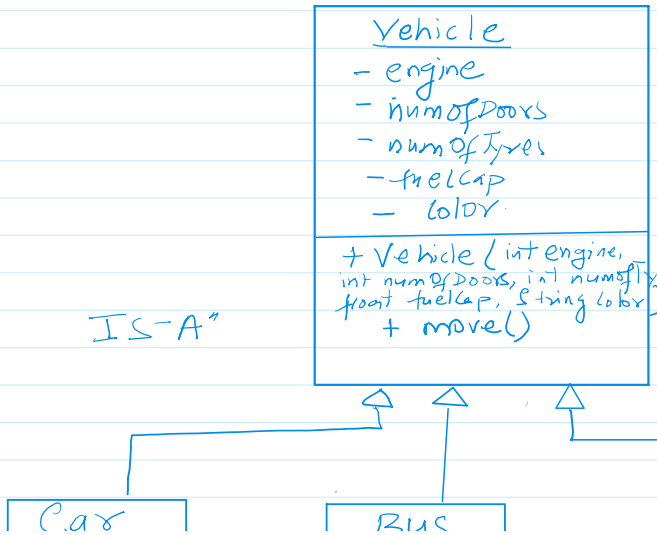
→ add()
→ get()
→ remove()

→ Collection Framework in Java.



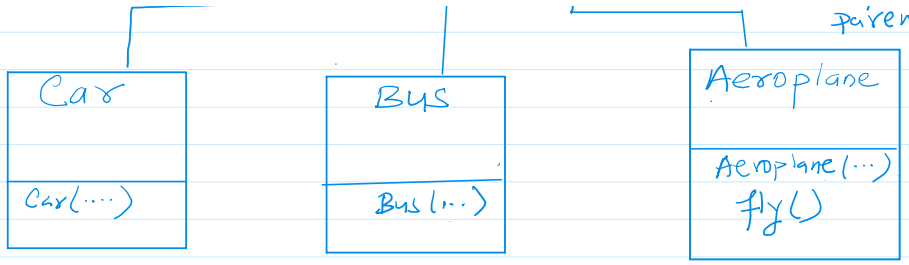
OOD (Object Oriented Design)

Re-Design of above inheritance hierarchy.



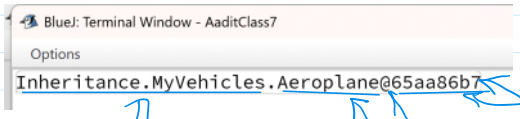
+ public (can be accessible from anywhere)
− private (Not inheritable)
• protected (when we inherit a class outside the package)
o default (only accessible in the same package)

(*) Constructor of parent/super class is not inheritable. Therefore, we need to call the parent class constructor using "super()". Reason is before child class



parent class constructor using "super()". Reason is before child class object exists, parent class object must exists.

(*) If class is not inheriting any other class, it is inheriting by default an Object class. Therefore we can say that 'Object' is the mother of all those classes which is not inheriting any other class. We can override the methods of Object class in our class.



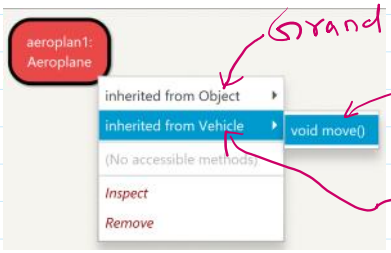
address of object on memory (RAM)

package name

separator

Class Name

This is appeared, because we haven't override the toString() method. Therefore it has executed the toString() method of Object class.



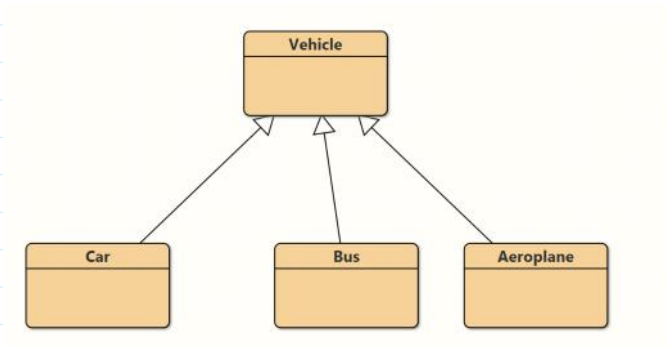
Grand Parent has the reference of object.

method inherited from parent class

parent class

This method has executed to print the reference (address) of the object.

It is called automatically by the System.out.print() method.



```java
package Inheritance.MyVehicles;


/**
 * Write a description of class Vehicle here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Vehicle
{
    private int engineNum;
    private int numberOfDoors;
    private int numberOfTyres;
    private String color;

    //parametrized constructor: it is used to initialize the instance variables
    public Vehicle(int engineNum, int numberOfDoors, int numberOfTyres, String color){
        this.engineNum = engineNum;
        this.numberOfDoors = numberOfDoors;
        this.numberOfTyres = numberOfTyres;
        this.color = color;
    }

    public void move(){
        System.out.print("\nVehicle is moving");
    }

    @Override
    public String toString(){
        return "[engineNum = "+ engineNum +
                    ", numberOfDoors = " + numberOfDoors +
                    ", numberOfTyres = " + numberOfTyres +
                    ", color = " + color + "]";
    }
}
```

```java
package Inheritance.MyVehicles;


/**
 * Write a description of class Car here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Car extends Vehicle
{
    private String typeOfCar;

    public Car(int engineNum, int numOfTyres, int numOfDoors, String color, String
typeOfCar){
        super(engineNum, numOfTyres, numOfDoors, color);
        this.typeOfCar = typeOfCar;
    }
```

```
    @Override
    public String toString(){
        return super.toString() + "[ typeOfCar = " + typeOfCar + "]";
    }

    @Override
    public void move(){
        System.out.print("\nCar is moving");
    }
}

package Inheritance.MyVehicles;


/**
 * Write a description of class Bus here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Bus extends Vehicle
{
    private String typeOfBus;

    public Bus(int engineNum, int numOfTyres, int numOfDoors, String color, String
typeOfBus){
        super(engineNum, numOfTyres, numOfDoors, color); //calling parent class
constructor.
        this.typeOfBus = typeOfBus;
    }

    @Override
    public String toString(){
        return super.toString()+"[ typeOfBus = " + typeOfBus + "]";
    }


    @Override
    public void move(){
        System.out.print("\nBus is moving");
    }
}


package Inheritance.MyVehicles;

/**
 * Write a description of class Aeroplane here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Aeroplane extends Vehicle
{
    private String typeOfAeroplane;

    public Aeroplane(int engineNum, int numOfTyres, int numOfDoors, String color,
```

```
    String typeOfAeroplane){
        super(engineNum, numOfTyres, numOfDoors, color);
        this.typeOfAeroplane = typeOfAeroplane;
    }

    @Override
    public String toString(){
        return super.toString() + "[ typeOfAeroplane = " + typeOfAeroplane + "]";
    }


    @Override
    public void move(){
        System.out.print("\nAeroplane is moving");
    }
}
```